

```
hide-note-list")
ument).on("keydown
ke) {
.3 == ke.keyCode) {
var xe = $(this),
Ce = xe.val();
if (is_blank(Ce))
var Ae = xe.parent()
je = T(Ae, "phot
ze = parseInt(Ae
R({
call: "comm str"
```

Listas en Python

Una de las estructuras de datos más poderosas y versátiles de Python. Aprende a crear, modificar y recorrer listas para almacenar colecciones de datos de forma eficiente.

PROGRAMACIÓN

ESTRUCTURAS DE DATOS

¿Qué es una Lista?

Una **lista** en Python es una colección ordenada y mutable que puede almacenar múltiples elementos en una sola variable. A diferencia de otros lenguajes, las listas en Python son increíblemente flexibles: pueden contener números, texto, booleanos o incluso mezclar tipos distintos en la misma lista.

 Las listas se definen usando `[]` y separando cada elemento con una coma.

```
mi_lista = [1, 2, 3, 4]
nombres = ["Ana", "Luis", "Pedro"]
mixta = [1, "hola", 3.5, True]
```

Como puedes ver, `mixta` combina un entero, un string, un float y un booleano sin ningún problema.

Acceder a los Elementos

Cada elemento de una lista tiene un **índice** que indica su posición. Python comienza a contar desde 0. También puedes usar índices negativos para acceder desde el final.

Índice positivo

```
mi_lista = [1, 2, 3, 4]
print(mi_lista[0]) # 1
print(mi_lista[1]) # 2
```

El primer elemento siempre está en el índice 0.

Índice negativo

```
print(mi_lista[-1]) # 4
print(mi_lista[-2]) # 3
```

Con -1 accedes al último elemento sin conocer el tamaño.



Modificar Elementos

¿Cómo funciona?

Las listas son **mutables**, lo que significa que puedes cambiar cualquier elemento después de crearla. Solo indicas el índice entre corchetes y asignas el nuevo valor con `=`.

Esto las diferencia de las *tuplas*, que son inmutables.

Ejemplo práctico

```
mi_lista = [1, 2, 3, 4]
```

```
# Cambiamos el elemento en posición 1
```

```
mi_lista[1] = 10
```

```
print(mi_lista)
```

```
# [1, 10, 3, 4]
```

✅ El valor `2` fue reemplazado por `10` en la posición 1.

Métodos Esenciales

Python incluye métodos incorporados para manipular listas fácilmente. Estos son los más utilizados en el día a día:

1

append()

```
mi_lista.append(5)
```

Agrega un elemento al **final** de la lista.

2

insert()

```
mi_lista.insert(1, 7)
```

Inserta un elemento en una **posición específica**.

3

remove()

```
mi_lista.remove(3)
```

Elimina la **primera ocurrencia** del valor indicado.

4

pop()

```
mi_lista.pop()
```

Elimina y retorna el **último elemento** de la lista.

Ordenar e Invertir

`sort()` — Ordenar

Ordena los elementos de la lista **en su lugar** (modifica la lista original) de menor a mayor. Para ordenar de mayor a menor, usa `sort(reverse=True)`.

```
numeros = [4, 1, 3, 2]
numeros.sort()
print(numeros)
# [1, 2, 3, 4]
```

`reverse()` — Invertir

Invierte el orden actual de los elementos, sin ordenarlos alfabética o numéricamente. Simplemente voltea la secuencia existente.

```
numeros = [1, 2, 3, 4]
numeros.reverse()
print(numeros)
# [4, 3, 2, 1]
```

⚠ Tanto `sort()` como `reverse()` modifican la lista original. Si necesitas conservarla, crea una copia primero con `lista.copy()`.

Recorrer una Lista con for

El bucle `for` es la forma más natural y pythónica de iterar sobre cada elemento de una lista, uno por uno, sin necesidad de manejar índices manualmente.

Sintaxis básica

```
nombres = ["Ana", "Luis", "Pedro"]
```

```
for nombre in nombres:  
    print(nombre)
```

```
# Ana  
# Luis  
# Pedro
```

Con índice usando enumerate()

```
for i, nombre in  
    enumerate(nombres):  
    print(i, nombre)
```

```
# 0 Ana  
# 1 Luis  
# 2 Pedro
```

 Usa `enumerate()` cuando necesites tanto el índice como el valor.

Listas por Comprensión

Las **listas por comprensión** (*list comprehensions*) son una de las características más elegantes de Python. Permiten crear listas nuevas en una sola línea, combinando un bucle y opcionalmente una condición.

1

Forma tradicional

```
cuadrados = []  
for x in range(5):  
    cuadrados.append(x**2)  
# [0, 1, 4, 9, 16]
```

2

Con comprensión

```
cuadrados = [x**2 for x in range(5)]  
# [0, 1, 4, 9, 16]
```

✅ La versión con comprensión es más corta, más legible y generalmente más rápida. ¡Es la forma preferida en Python profesional!

Comprensiones con Condición

Puedes agregar un filtro `if` al final de una comprensión para incluir solo los elementos que cumplan una condición. Esto las hace aún más poderosas:

Solo números pares

```
pares = [x for x in range(10) if x % 2 == 0]
print(pares)
# [0, 2, 4, 6, 8]
```

Palabras largas

```
palabras = ["sol", "computadora", "paz", "programación"]
largas = [p for p in palabras if len(p) > 4]
# ['computadora', 'programación']
```

Transformar y filtrar

```
negativos = [-3, -1, 2, 5, -4]
positivos = [x for x in negativos if x > 0]
# [2, 5]
```

Longitud y Resumen General

len() — Longitud

La función `len()` devuelve el número de elementos que contiene una lista. Es fundamental para validaciones, bucles y control de flujo.

```
mi_lista = [1, 10, 3, 4]
print(len(mi_lista))
# 4
```

 `len()` no es un método de la lista — es una función integrada de Python que funciona con muchos otros tipos de datos.

Todo lo que aprendiste hoy

01

Crear listas

Con `[]`, admiten tipos mixtos.

02

Acceder y modificar

Por índice positivo o negativo.

03

Métodos clave

`append`, `insert`, `remove`, `pop`, `sort`, `reverse`.

04

Iterar y comprensiones

Con `for` y sintaxis compacta `[x for x in ...]`.

EJEMPLO:

```
=====

EJEMPLO COMPLETO DE LISTAS EN PYTHON

=====

1. CREAR UNA LISTA

mi_lista = [10, 20, 30, 40] print("Lista inicial:", mi_lista)

2. AGREGAR ELEMENTOS

mi_lista.append(50) # Agrega al final print("Después de append(50):", mi_lista)

mi_lista.insert(2, 25) # Inserta 25 en la posición 2 print("Después de insert(2, 25):", mi_lista)

3. ACCEDER A ELEMENTOS

print("Primer elemento:", mi_lista[0]) print("Último elemento:", mi_lista[-1])

4. MODIFICAR ELEMENTOS

mi_lista[1] = 15 print("Después de modificar posición 1:", mi_lista)

5. ELIMINAR ELEMENTOS

mi_lista.remove(25) # Elimina el valor 25 print("Después de remove(25):", mi_lista)

eliminado = mi_lista.pop() # Elimina el último print("Elemento eliminado con pop():", eliminado) print("Lista
actual:", mi_lista)

6. ORDENAR LA LISTA

mi_lista.sort() print("Lista ordenada:", mi_lista)

7. INVERTIR LA LISTA

mi_lista.reverse() print("Lista invertida:", mi_lista)

8. RECORRER LA LISTA CON FOR

print("Recorriendo lista:") for elemento in mi_lista: print(elemento)

9. LONGITUD DE LA LISTA

print("Cantidad de elementos:", len(mi_lista))

10. LISTA DE NOMBRES

nombres = ["Ana", "Luis", "Carlos"] print("Lista de nombres:", nombres)

11. AGREGAR MÁS NOMBRES

nombres.append("María") print("Nombres actualizados:", nombres)

12. LISTAS POR COMPRENSIÓN

cuadrados = [x**2 for x in range(1, 6)] print("Lista de cuadrados:", cuadrados)

13. SUMA DE ELEMENTOS

suma = sum(mi_lista) print("Suma de elementos de mi_lista:", suma)

14. COMPROBAR SI UN ELEMENTO EXISTE

if 30 in mi_lista: print("El número 30 sí está en la lista") else: print("El número 30 no está en la lista")

15. LISTA MIXTA

mixta = [1, "Python", True, 3.14] print("Lista mixta:", mixta)

=====

FIN DEL EJEMPLO

=====
```